

Web Development with HTML, CSS and JavaScript

Notes[Chapter # 5]

Posted in **9 Computer Science**



These lecture notes are prepared by **Salman Ahmad**, who is both a **teacher** and a **web developer**. The purpose of this material is to make the topic clear and useful for students of Class 9 Computer Science.

Table of Contents

- Web Development
- Why Learn Web Development?
- Basic Components of Web Development
- Front-End Development
- Back-End Development
- Full-Stack Development

Setting Up a Development Environment

Text Editors

Web Browsers

Basic Structure of HTML

HTML Tags

Types of HTML Tags

Creating Content with HTML

Headings

Importance of Headings

Paragraphs

Links

Images

Lists

Ordered List

Unordered List

Creating Tables in HTML

HTML Comments

Styling with CSS

Integrating CSS in HTML

Creating Layout and Organizing Content

JavaScript

Variables

Data Types

Functions

Functions with Parameters

MCQs

MCQs for testing

FAQs

Web Development

Web development is the process of creating websites and web applications using programming languages and web technologies.

Example: Creating a website using HTML, CSS, and JavaScript that users can access through a web browser.

Why Learn Web Development? (50% Important – Long Question)

Learning web development is useful for developing digital literacy, building career opportunities, improving problem-solving skills, expressing creativity, and supporting entrepreneurship.

1. Digital Literacy

Web development helps students understand **how websites and web applications are created and work**. It improves their ability to use and understand modern digital technologies.

Example: Creating a simple webpage using HTML and CSS.

2. Career Opportunities

Web development provides **many career opportunities** in the technology field.

Examples: Web developer, front-end developer, back-end developer, and full-stack developer.

3. Problem Solving

Web development improves **problem-solving skills** because developers need to identify errors, find solutions, and create working programs.

Example: Finding and fixing an error in a website's code.

4. Creativity

Web development allows people to **express their ideas creatively** by designing websites, layouts, animations, and interactive features.

Example: Designing a personal portfolio website.

5. Entrepreneurship

Web development allows people to **create websites and online services for their own businesses or clients**.

Example: Creating an online store to sell products.

Basic Components of Web Development

(90% Important – Long Question)

Web development has different components that work together to create complete and functional websites and web applications.

1. Front-End Development

Front-end development focuses on the **part of a website that users see and interact with**. It includes the webpage layout, text, images, buttons, menus, forms, and other interactive elements.

The main technologies used in front-end development are **HTML, CSS, and JavaScript**.

HTML

HTML (HyperText Markup Language) is used to **create the structure and content of a webpage**.

It defines elements such as:

- Headings
- Paragraphs
- Images
- Links
- Tables
- Forms
- Buttons

Example:

```
<h1>My Website</h1>
<p>Welcome to my website.</p>
```

Here, HTML creates the **heading and paragraph** of the webpage.

CSS

CSS (Cascading Style Sheets) is used to **style and design a webpage**. It controls the appearance and layout of HTML elements.

CSS can control:

- Colors

- Fonts
- Sizes
- Spacing
- Borders
- Layout
- Responsive design

Example:

```
h1 {  
  color: blue;  
  font-size: 30px;  
}
```

Here, CSS changes the **color and size** of the heading.

JavaScript

JavaScript is a programming language used to add **interactivity and dynamic behavior** to webpages.

It can be used to:

- Respond to button clicks
- Validate forms
- Change webpage content
- Create animations
- Perform calculations
- Communicate with a server

Example: When a user clicks a button, JavaScript can display a message.

2. Back-End Development

Back-end development focuses on the **server-side part of a website or web application**. It processes requests, manages data, and performs tasks that users do not directly see.

The main components include **web servers, databases, and back-end programs**.

Web Servers

A **web server** is a computer system or software that **receives requests from users and sends webpages or other resources back to their browsers**.

Example: When you open a website, your browser sends a request to the web server, and the server sends the required webpage back to your browser.

Databases

A **database** is used to **store and manage data** required by a website or web application.

It can store information such as:

- User accounts
- Passwords
- Products
- Orders
- Blog posts
- Comments

Example: An online store uses a database to store product names, prices, and customer orders.

Back-End

The **back-end** is the part of a web application that **runs on the server and handles the logic, data, and requests**.

Back-end programming languages and technologies include **Python, PHP, Java, JavaScript, Node.js, ReactJs, Ruby**, and others.

Example: When a user logs into a website, the back-end receives the login information, checks it against the database, and returns the appropriate result.

3. Full-Stack Development

Full-stack development involves working on **both the front-end and back-end** of a website or web application.

A **full-stack developer** can work with:

- **Front-end:** HTML, CSS, JavaScript
- **Back-end:** Server-side programming
- **Databases:** Storing and managing data
- **Web servers:** Handling requests and responses

Example: A full-stack developer can create an online shopping website, including its webpage design, login system, product database, and server-side functionality.

Note: In a login system, the **front-end** provides the login form, the **web server** receives the request, the **back-end** processes the login details, and the **database**

verifies the user's information. A **full-stack developer** can develop all these components together.

Setting Up a Development Environment

A **development environment** is the set of tools used by a developer to **write, test, and run web projects**.

1. Text Editors

A **text editor** is a software tool used to **write and edit code**.

Examples: Visual Studio Code, Notepad++, and Sublime Text.

Example: A developer can use Visual Studio Code to write HTML, CSS, and JavaScript code.

2. Web Browsers

A **web browser** is software used to **open, view, and test websites**.

Examples: Google Chrome, Microsoft Edge, and Mozilla Firefox.

Note: After writing HTML and CSS code, a developer can open the webpage in a web browser such as Chrome to check how it looks and works.

Basic Structure of HTML

An HTML document has a basic structure that helps the browser understand and display the webpage.

- **<!DOCTYPE html>** tells the browser that the document uses HTML5.
- **<html>** is the main element that contains the entire HTML document.
- **<head>** contains information about the webpage, such as the title and metadata.
- **<title>** defines the title displayed in the browser tab.
- **<body>** contains the visible content of the webpage, such as headings, paragraphs, images, and links.

Example of Basic HTML Structure

```
<!DOCTYPE html>
<html>
<head>
```

```
<title>ComputerWithSalman</title>
</head>
<body>

<h1>Welcome to ComputerWithSalman</h1>
<p>ComputerWithSalman is a YouTube channel for Computer Science learning.

<h2>SalmanBlogs.com</h2>
<p>SalmanBlogs.com is a website providing Computer Science notes
and learning resources.</p>

</body>
</html>
```

HTML Tags

HTML tags are special keywords written inside **angle brackets (< >)** that tell the browser how to structure and display content on a webpage.

HTML tags are mainly divided into **two types: Pair Tags and Singular Tags**.

Types of HTML Tags

1. Pair Tags

Pair tags have both an **opening tag** and a **closing tag**. The content is written between them.

Example:

```
<p>Welcome to ComputerWithSalman</p>
```

Here:

- **<p>** → Opening tag
- **</p>** → Closing tag
- **Welcome to ComputerWithSalman** → Content

More examples: `<h1>...</h1>`, `<title>...</title>`, `<strong>...</strong>`

2. Singular Tags

Singular tags do not require a closing tag. They are used for elements that do not contain content.

Example:

```
<br>
```

The **
** tag is used to create a **line break**.

More examples: `<br>`, `<hr>`, `<img>`, `<input>`

Note: Other names for Pair and Singular tags.

Pair / Container / Paired → Opening + Closing tag

Singular / Empty / Void / Unpaired → No closing tag

Creating Content with HTML

HTML is used to **create and structure content on webpages**. It provides different tags for headings, paragraphs, images, links, lists, and other elements.

Headings

HTML provides **six levels of headings**, from `<h1>` to `<h6>`. The `<h1>` heading is the most important and largest by default, while `<h6>` is the least important and smallest.

Example of Headings

```
<h1>Computer Science</h1>
<h2>Web Development</h2>
<h3>HTML</h3>
<h4>HTML Tags</h4>
<h5>Pair Tags</h5>
<h6>Examples of Pair Tags</h6>
```

Output

Computer Science

Web Development

HTML

HTML Tags

Pair Tags

Examples of Pair Tags

Importance of Headings

1. Organizing Content

Headings divide a webpage into **different sections and subsections**, making the content easier to read and understand.

Example:

```
Computer Science    → H1
  Web Development   → H2
    HTML            → H3
      HTML Tags     → H4
```

2. SEO

Headings help **search engines understand the structure and topics of a webpage**. Properly organized headings can help search engines understand the content more clearly and ranking your content higher in search results.

Example: Using `<h1>` for the main topic and `<h2>` and `<h3>` for related sections.

3. Consistent Formatting

Using heading tags provides **consistent formatting and structure** throughout a webpage.

All `<h2>` headings can be given the same font size, color, and style using CSS, making the webpage look organized and professional.

Note: HTML provides six heading levels: `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, and `<h6>`. Use headings in a logical order to organize webpage content clearly.

Paragraphs

The `<p>` tag is used to create a **paragraph of text** on a webpage.

Example:

```
<p>Welcome to ComputerWithSalman, a YouTube channel for learning Computer Sci
```

Output:

Welcome to ComputerWithSalman, a YouTube channel for learning Computer Science.

Links

The `<a>` tag is used to create **links** that allow users to visit another webpage or website.

Example:

```
<a href="https://salmanblogs.com">Visit SalmanBlogs.com</a>
```

Output:

[Visit SalmanBlogs.com](https://salmanblogs.com)

Images

The `<img>` tag is used to **display an image** on a webpage. It is a **singular (empty) tag**, so it does not require a closing tag.

Example:

```

```

- `src` specifies the **location or name of the image**.
- `alt` provides **alternative text** that describes the image.

Lists

Lists are used to **display related items in an organized way**. HTML provides two common types of lists: **ordered and unordered lists**.

Ordered List

An **ordered list** displays items in a specific order, usually using **numbers**. It is created using the `<ol>` tag.

Example:

```
<ol>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ol>
```

Output:

1. HTML
2. CSS
3. JavaScript

Unordered List

An **unordered list** displays items using **bullet points** without a specific order. It is created using the `<ul>` tag.

Example:

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

Output:

- HTML
- CSS
- JavaScript

Creating Tables in HTML

HTML provides table tags to display data in **rows and columns**. Tables are useful for presenting information in an organized and easy-to-read format.

1. <table> Tag

The <table> tag is used to **create a table** in HTML. It contains all the rows and cells of the table.

Example:

```
<table>  
  ...  
</table>
```

2. <tr> Tag

The <tr> tag is used to create a **table row**. Each row can contain heading cells or data cells.

Example:

```
<tr>  
  ...  
</tr>
```

3. <td> Tag

The <td> tag is used to create a **table data cell**. It contains the actual data in a table.

Example:

```
<td>Ali</td>
```

4. <th> Tag

The <th> tag is used to create a **table heading cell**. It is normally used for headings such as Name, Class, or Marks.

Example:

```
<th>Name</th>
```

Complete Example

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Class</th>
    <th>Marks</th>
  </tr>

  <tr>
    <td>Ali</td>
    <td>9</td>
    <td>85</td>
  </tr>

  <tr>
    <td>Ahmed</td>
    <td>9</td>
    <td>90</td>
  </tr>
</table>
```

Output

Name	Class	Marks
Ali	9	85
Ahmed	9	90

HTML Comments

HTML comments are used to add **notes or explanations inside HTML code**.

Comments are not displayed on the webpage by the browser.

They are useful for **explaining code, organizing sections, and temporarily disabling code**.

Syntax of HTML Comments

HTML comments are written between `<!--` and `-->`.

Example:

```
<!-- This is an HTML comment -->
```

Example

```
<!-- Main Heading -->
<h1>ComputerWithSalman</h1>

<!-- Welcome Message -->
<p>Welcome to our website.</p>
```

The comments will be visible in the **HTML source code**, but they will **not appear on the webpage**.

Easy to remember:

```
<!-- Comment --> → Comment in HTML
```

Styling with CSS

CSS (Cascading Style Sheets) is used to **style and design HTML webpages**. It controls the appearance and layout of HTML elements, such as colors, fonts, sizes, spacing, borders, and backgrounds.

Basic Structure of CSS

A CSS rule consists of a **selector**, **property**, and **value**.

Syntax

```
selector {
  property: value;
}
```

Example

```
h1 {  
  color: blue;  
  font-size: 30px;  
}
```

- **Selector:** `h1` → selects the HTML heading.
- **Property:** `color` → specifies what aspect will be changed.
- **Value:** `blue` → specifies the color to apply.
- **Declaration:** `color: blue;` → property and its value together.

Easy to remember:

Selector → Selects the HTML element

Property → Specifies what to change

Value → Specifies what value to apply

Declaration → Property + Value

Integrating CSS in HTML

CSS can be added to an HTML document in three common ways:

1. Inline CSS
2. Internal CSS
3. External CSS

1. Inline CSS

Inline CSS is written **directly inside an HTML element** using the `style` attribute.

Example:

```
<h1 style="color: blue;">ComputerWithSalman</h1>
```

Here, the `style` attribute applies CSS directly to the `<h1>` element.

Use: Suitable for applying a style to a specific HTML element.

2. Internal CSS

Internal CSS is written inside the `<style>` tag, usually within the `<head>` section of an HTML document.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <style>
    h1 {
      color: blue;
    }

    p {
      font-size: 18px;
    }
  </style>
</head>

<body>
  <h1>ComputerWithSalman</h1>
  <p>Learn Computer Science with easy explanations.</p>
</body>
</html>
```

Use: Suitable for styling a single HTML webpage.

3. External CSS

External CSS is written in a **separate CSS file** and connected to the HTML document using the `<link>` tag.

HTML file:

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="style.css">
</head>

<body>
```

```
<h1>ComputerWithSalman</h1>
<p>Learn Computer Science with easy explanations.</p>
</body>
</html>
```

CSS file (style.css):

```
h1 {
  color: blue;
}

p {
  font-size: 18px;
}
```

Use: Suitable for styling multiple webpages using the same CSS file.

Easy to remember:

Inline CSS → CSS inside the HTML element

Internal CSS → CSS inside the `<style>` tag

External CSS → CSS in a separate `.css` file

Creating Layout and Organizing Content

CSS is used to create the **layout of a webpage and organize its content**. It helps arrange headings, paragraphs, images, menus, cards, and other elements in a clear and attractive way.

HTML provides elements such as `<div>` and `<section>` to organize content, while CSS provides layout techniques such as **Grid** and **Flexbox**.

1. `<div>` and `<section>`

`<div>` Tag

The `<div>` tag is a **container** used to group HTML elements together. It does not have any special meaning by itself and is commonly styled using CSS.

Example:

```
<div>
  <h2>Computer Science</h2>
  <p>Learn Computer Science with easy explanations.</p>
</div>
```

A `<div>` can be used to group related content and apply CSS styles to the group.

`<section>` Tag

The `<section>` tag is used to define a **specific section of related content** on a webpage.

Example:

```
<section>
  <h2>Web Development</h2>
  <p>HTML and CSS are used to create webpages.</p>
</section>
```

Easy to remember:

<div> → General container for grouping content

<section> → Groups a specific section of related content

CSS Grid

CSS Grid is a CSS layout system used to arrange elements in **rows and columns**. It is useful for creating layouts such as **cards, image galleries, and webpage sections**.

Example

HTML:

```
<div class="container">
  <div>HTML</div>
  <div>CSS</div>
  <div>JavaScript</div>
  <div>Python</div>
  <div>Networking</div>
```

```
<div>Database</div>
</div>
```

CSS:

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
```

Here, `grid-template-columns: 1fr 1fr 1fr;` creates **three equal-width columns**.

Using auto in CSS Grid

The `auto` value allows the browser to automatically determine the size of a grid column based on the available space and content.

Example:

```
.container {
  display: grid;
  grid-template-columns: auto auto auto;
  gap: 10px;
}
```

Here, `auto auto auto` creates **three columns**, and the browser automatically determines their sizes.

Easy to remember:

CSS Grid → Arranges elements in rows and columns

1fr 1fr 1fr → Three equal-width columns

auto auto auto → Three columns with automatically determined sizes

CSS Flexbox

CSS Flexbox is a CSS layout system used to arrange elements in a **row or column** and control their alignment and spacing.

It is useful for navigation menus, buttons, cards, and other elements that need to be arranged in a flexible way.

Example:

HTML:

```
<div class="container">
  <div>HTML</div>
  <div>CSS</div>
  <div>JavaScript</div>
</div>
```

CSS:

```
.container {
  display: flex;
  gap: 20px;
}
```

In this example, the items are arranged in a **row** with space between them.

Flexbox vs Grid

Both **Flexbox** and **CSS Grid** are CSS layout systems, but they are mainly used for different types of layouts.

- **Flexbox:** Mainly used for arranging elements in **one direction** — a row or a column.
- **Grid:** Used for arranging elements in **rows and columns**.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flexbox vs Grid</title>

  <style>
    /* Common styling */
```

```
.box {
  padding: 20px;
  border: 2px solid black;
  text-align: center;
}

/* Flexbox - One Direction */
.flexbox {
  display: flex;
  flex-direction: row;
  gap: 10px;
  margin-bottom: 30px;
}

/* Grid - Rows AND Columns */
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
</style>
</head>

<body>

  <h2>Flexbox</h2>

  <div class="flexbox">
    <div class="box">Item 1</div>
    <div class="box">Item 2</div>
    <div class="box">Item 3</div>
  </div>

  <h2>Grid</h2>

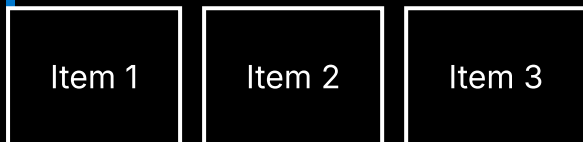
  <div class="grid">
    <div class="box">Item 1</div>
    <div class="box">Item 2</div>
    <div class="box">Item 3</div>
    <div class="box">Item 4</div>
    <div class="box">Item 5</div>
```

```
<div class="box">Item 6</div>
</div>

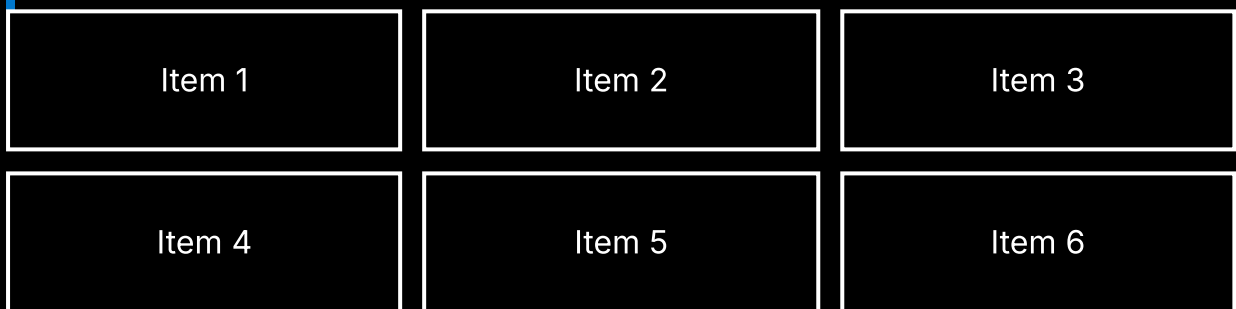
</body>
</html>
```

Output

Flexbox



Grid



Easy to remember:

Flexbox → One Direction → Row or Column

Grid → Two Dimensions → Rows and Columns

JavaScript

JavaScript is a programming language used to add **interactivity and dynamic behavior** to webpages. It can respond to user actions, change webpage content, perform calculations, and validate forms.

Basic Syntax of JavaScript

JavaScript statements are written inside `<script>` tags when JavaScript is included directly in an HTML document.

Example:

```
<script>
  let name = "Salman";
  console.log(name);
</script>
```

Here, `let` creates a variable and `console.log()` displays its value in the browser console.

Simple JavaScript Example

```
<script>
  alert("Welcome to ComputerWithSalman!");
</script>
```

This displays a welcome message in an alert box.

Variables

A **variable** is used to store data in a program. JavaScript provides three keywords for declaring variables:

- `let`
- `var`
- `const`

1. `let`

`let` is used to declare a variable whose value can be changed later.

```
let age = 15;
age = 16;
```

The value of `age` changes from 15 to 16.

2. `var`

`var` is an older way of declaring variables in JavaScript. Its value can also be changed.

```
var name = "Ali";
```

```
name = "Ahmed";
```

`var` is generally avoided in modern JavaScript when `let` or `const` can be used.

3. `const`

`const` is used to declare a variable whose value cannot be reassigned.

```
const pi = 3.14159;
```

The value of `pi` cannot be reassigned.

Easy to remember:

let → Value can be changed

var → Older way of declaring variables

const → Value cannot be reassigned

Data Types

A **data type** defines the type of value stored in a variable.

Common JavaScript data types include:

1. String

A string stores text.

```
let name = "Salman";
```

2. Number

A number stores numeric values.

```
let marks = 85;
```

3. Boolean

A Boolean stores either `true` or `false`.

```
let passed = true;
```

4. Undefined

A variable has the value `undefined` when it has been declared but no value has been assigned.

```
let result;
```

5. Null

`null` represents an intentionally empty or absent value.

```
let student = null;
```

Easy to remember:

String → Text

Number → Numeric value

Boolean → `true` OR `false`

Undefined → Declared but no value assigned

Null → Intentionally empty or absent value

Functions

A **function** is a block of code designed to perform a specific task. A function can be called whenever that task needs to be performed.

Basic Syntax

```
function functionName() {  
    // code  
}
```

Simple Function – Welcome Message

```
function welcome() {  
    alert("Welcome to ComputerWithSalman!");  
}  
  
welcome();
```

Explanation

- `function` → keyword used to create a function.
- `welcome` → name of the function.
- `{ }` → contains the code of the function.
- `alert()` → displays the message.
- `welcome();` → calls the function and executes its code.

Functions with Parameters

A **parameter** is a value received by a function so that the function can use it while performing its task.

1. Function with a Single Parameter

```
function welcome(name) {  
    alert("Welcome " + name);  
}  
  
welcome("Salman");
```

Output:

Welcome Salman

Here, `name` is the **parameter** and `"Salman"` is the **argument** passed to the function.

2. Function with Multiple Parameters

A function can receive more than one parameter.

```
function add(a, b) {  
    let result = a + b;
```

```
    alert(result);  
}  
  
add(10, 20);
```

Output:

30

Here:

- **a** → first parameter
- **b** → second parameter
- **10 and 20** → arguments
- **a + b** → adds the two values

Easy to Remember

Function without parameters:

```
function welcome() {  
    alert("Welcome!");  
}
```

Function with one parameter:

```
function welcome(name) {  
    alert("Welcome " + name);  
}
```

Function with multiple parameters:

```
function add(a, b) {  
    return a + b;  
}
```

Easy to remember:

Function → Performs a specific task

Parameter → Input of a function

Argument → Actual value passed to the function

Multiple Choice Questions (MCQs) on Entrepreneurship in the Digital Age

1. What is web development?

- a. Creating computer games only
- b. Creating websites and web applications
- c. Repairing computers
- d. Installing operating systems

Answer: b. Creating websites and web applications

2. Which technology is mainly used to create the structure of a webpage?

- a. CSS
- b. JavaScript
- c. HTML
- d. Python

Answer: c. HTML

3. Which technology is used to style the appearance of a webpage?

- a. HTML
- b. CSS
- c. JavaScript
- d. SQL

Answer: b. CSS